

Informe Trabajo Práctico N°2

72.11 Sistemas Operativos

Grupo 13

Lucila Borinsky

Santiago Diaz Sieiro

Toribio Viton Sconza

Legajo: 63039

Legajo: 63058

Legajo: 64275



Índice

1. Introducción	2
2. Decisiones tomadas durante el desarrollo	4
3. Instrucciones de compilación y ejecución	5
4. Limitaciones	6
5. Problemas encontrados y cómo se solucionaron	7
6. Cita de fragmento de códigos reutilizados	8
7. Conclusión	9

1. Introducción

En este segundo trabajo práctico construimos un **kernel monolítico de 64-bits**, sobre la base del proyecto especial de Arquitectura de Computadoras, con el objetivo de implementar los principales mecanismos de **gestión y administración de recursos**: memoria física, multitarea preemptiva, IPC, sincronización y una **shell** de usuario que orquesta todas las pruebas.

El informe profundiza en cómo vamos a desarrollar las decisiones de diseño tomadas, como compilar y ejecutar el Trabajo y como probar que las implementaciones realizadas funcionen correctamente

2. Decisiones tomadas durante el desarrollo

2.1 Gestión de memoria física

Implementación dual mediante compilación condicional (**#ifdef BUDDY**) que permite elegir entre dos gestores de memoria:

- **Buddy System**: División recursiva de bloques en potencias de 2, optimizado para fragmentación mínima
- **Heap4**: Gestor de memoria con lista enlazada de bloques libres, más simple pero eficiente para asignaciones pequeñas

Ambas implementaciones comparten la interfaz definida en **memory.h** y pasan los tests de memoria tanto en Kernel como en Userland.

2.2 Procesos, cambio de contexto y scheduler

- **Multitarea**: Implementada mediante una lista circular de PCBs (Process Control Blocks) que mantiene el estado de cada proceso, incluyendo su stack, registros y recursos asociados.
- **Round Robin con prioridades**: Scheduler que asigna quantum de CPU proporcionales a la prioridad del proceso (1-9), donde cada nivel de prioridad determina cuántos quantum consecutivos recibe el proceso antes de ceder la CPU.
- **Context switch**: Implementado mediante `switcherInterrupt()` que: Preserva el contexto actual (r15-r12, rbx, rbp, rsp), actualiza el PCB del proceso actual, carga el contexto del siguiente proceso y utiliza `iretq` para restaurar el estado de usuario

2.3 IPC y sincronización

- **Pipes**: Implementación de buffers circulares de 4KB con semáforos para control de acceso: Mutex para exclusión mutua, semáforos empty/written para sincronización productor-consumidor y operaciones bloqueantes read/write con manejo de EOF
- **Semáforos**: Implementación POSIX-like con: Cola FIFO de procesos bloqueados, Spinlock para exclusión mutua, Operaciones atómicas wait/post y Wrappers en libc para transparencia usuario/kernel

2.4 Shell de usuario (sh)

- **Parser:** Implementa un intérprete de comandos que soporta: Comandos simples con argumentos, Procesos en background (&), Pipelines de dos procesos (|), Redirección de entrada/salida (> y <) ??? nose si va esto
- **Modelo fork-exec:**
 - Built-ins con ps, mem, help, clear ejecutados en el proceso actual
 - Manejo de señales: Ctrl+C envía SIG_KILL al proceso en primer plano
 - Ctrl+D: Envía EOF al proceso actual

2.5 Drivers y capa de abstracción

Reutilizamos los *drivers* de **video** y **teclado** del TP de Arquitectura, encapsulados detrás de *syscalls* para respetar la separación *kernel/user*.

2.6 Herramientas de calidad y CI

- **Docker oficial ITBA** (*agodio/itba-so-multi-platform:3.0*) como única imagen de construcción.
- **GitHub Actions:** make all, make buddy, valgrind --leak-check=full, pvs-studio-analyzer, y los tests testMM, testProcesses, testPriority, testSem y test_no_synchro??

3. Instrucciones de compilación y ejecución

Clonar proyecto y posicionarse en la raíz

`git clone <repo> && cd TP2_SO`

`docker pull agodio/itba-so-multi-platform:3.0`

`docker run --rm -it -v "$PWD":/tp2 -w /tp2 \agodio/itba-so-multi-platform:3.0 bash`

Compilar con nuestro memory manager (defecto)

***`./compile.sh`** *# genera Image/x64BareBonesImage.qcow2**

***`./run.sh`** *# arranca QEMU con 512 MiB RAM**

Compilar con Buddy

Descomentar el: ***`#ifdef BUDDY`***

Todos los binarios se colocan en bin/. El ***`./compile.sh`*** produce solamente un warning que es un falso positivo con -Wall -Wextra.

4. Pasos a seguir para demostrar el funcionamiento de cada uno de los requerimientos

- **Memory Manager:** Se puede testear el funcionamiento del Memory Manager con el comando *TestMM* desde la Shell seguido por un número. Este número va a ser la cantidad máxima de bytes que asigne el Memory Manager. También se puede correr el comando *memState* que imprime el estado de la memoria.
- **Procesos y Scheduler:** Al correrlo en background, se puede ir poniendo el comando *ps* para ver cómo se van creando, finalizando y cambiando el estado de los procesos. También se puede probar el cambio de prioridades utilizando el comando *nice* seguido por el PID del proceso y un número entre 1 y 10. Luego, utilizar el comando *ps* para ver que efectivamente se modificó su prioridad. Entre otros se encuentran: kill, loop. Otro interesante es el *&* se aplica luego de cualquier comando mencionado y cuenta con el mecanismo para determinar si va a ceder o no al foreground al proceso que se ejecuta.
- **Sincronización:** Para testear el funcionamiento de los semáforos, se puede usar el comando *TestSem* de la siguiente manera: *Test <n> <use_sem> <exp_val>* donde *n* puede ser cualquier número positivo e indica la cantidad de sumas y restas que va a realizar el test. En el caso de *use_sem*, este indica si queremos que se usen o no semáforos durante el testeo. En caso de ser 0, no se activan los semáforos y el resultado del test puede ser cualquiera. En caso de ser 1, se activan los semáforos y el resultado del test debe ser 0, que es el comportamiento esperado. Otra manera de probar que la sincronización funciona correctamente, es mediante el comando *phylo* ya que internamente utiliza semáforos para solucionar el problema de los filósofos
- **Pipes:** Para ver el correcto funcionamiento de los pipes se puede usar el *wc*, *filter* y *cat*

5. Limitaciones

- **Tamaño fijo de heap por proceso:** `PROCESS_HEAP_SIZE` = 2^{16} bytes
- **Buddy System limitado** a 64MB de memoria total: `HEAP_SIZE` = 2^{26} bytes
- No maneja **memoria virtual**
- **Máximo de 10** procesos en espera por semáforo (`waitingPCBs[10]`)
- Prioridades **fijas** entre 1-9, sin prioridades dinámicas
- **Pipes limitados** a 4KB de buffer: `BUFFER_SIZE` = 40
- **Máximo de 20** semáforos simultáneos: `INITIAL_CAPACITY` = 20
- Pipeline limitado a **2 procesos**, no soporta múltiples pipes
- No implementa redirección **múltiple** (`>>`, `2>`, etc)

6. Problemas encontrados y soluciones

- **Pipes**

Los principales problemas encontrados durante el desarrollo estuvieron ligados principalmente a la implementación de *pipes*. Si bien el desarrollo de ambos managers de memoria tuvo muchos problemas al inicio, la manipulación de pipes desde la entrada estándar. El principal inconveniente fue que cuando se quería usar alguna función de *input* como *getchar*, el pipe leía infinitamente caracteres *null terminated*, lo que provocaba que la *shell*, fuera inusable ya que no aceptaba ningún otro *input* y el cursor de la terminal avanzaba indefinidamente sin manera de ponerle freno.

Para poder solucionar este problema tuvimos que hacer lo siguiente:

- Emplear semáforos de forma eficiente para poder sincronizar con otros procesos la lectura desde la entrada estándar de manera ordenada y que no corrompe datos que se encontraran dentro del pipe
- Cerciorar por medio de chequeos que el pipe tenga realmente datos que hayan sido escritos a él y que no esté vacío.

- **Función getUniform**

Por alguna razón que desconocemos una de las funciones provistas por la cátedra dentro de una de las librerías que asisten al testeo, en específico getUniform, cuya función es retornar un valor numérico pseudoaleatorio solo devuelve el número 0 (cero). En consecuencia, para circunvalar esta limitación, hicimos uso de una función propia que suplanta a la susodicha en su trabajo de generar dichos números.

7. Citas de fragmentos de código reutilizados

1. Algoritmo **Buddy** adaptado de *Linux 0.11* (mm/buddy.c).
2. **Reader-writer spinlock** inspirado en *Operating Systems: Three Easy Pieces*, Cap. 29
3. **Github** brindado por la cátedra

8. Conclusión

A lo largo de este trabajo se desarrolló un kernel monolítico de 64 bits que integra las funcionalidades fundamentales de un sistema operativo, incluyendo gestión de memoria física, multitarea con cambio de contexto, mecanismos de IPC, sincronización mediante semáforos y una shell interactiva que permite probar todas las funcionalidades implementadas. Las decisiones de diseño tomadas, como la implementación condicional de dos gestores de memoria (Buddy System y Heap4) y el uso de un scheduler Round Robin con prioridades, permitieron una solución flexible y modular que se adapta a diferentes escenarios de prueba y uso.

Si bien el sistema cumple con los requerimientos establecidos y los tests diseñados validan el correcto funcionamiento de los distintos componentes, se identificaron ciertas limitaciones estructurales, como la ausencia de memoria virtual, el tamaño fijo de heap por proceso y restricciones en la cantidad de procesos o semáforos simultáneos. Además, se presentaron desafíos específicos durante el desarrollo, especialmente en la implementación de pipes, que fueron resueltos mediante mecanismos de sincronización y control de flujo adecuados.

En definitiva, el proyecto no solo permitió aplicar los conocimientos teóricos adquiridos, sino también enfrentar y resolver problemas propios del desarrollo de bajo nivel, dejando sentadas las bases para futuras mejoras y extensiones del sistema operativo.